

Research: Suitable Tech-Stacks & Frameworks for Meeting Scheduling Platform

Evaluating modern architecture approaches, framework choices, and technology recommendations for the platform

Prepared: August 2026

Table of Contents

1. Scope & Methodology
 2. Stack-by-Stack Analysis
 - 2.1 T3 Stack (Next.js + React + TypeScript)
 - 2.2 Cal.diy Architecture (Production Open-Source)
 - 2.3 Modern SaaS Stack (Supabase + Next.js)
 - 2.4 Enterprise-Grade Stack (Node.js + Express)
 3. Side-by-Side Framework Comparison
 4. Mobile Development Framework Analysis
 5. Real-Time & Background Job Systems
 6. Recommended Stack for Your Platform
 7. Conclusion
-

1. Scope & Methodology

Four technology stack approaches were studied to identify the optimal architecture for your unified scheduling platform that bridges polling + 1-on-1 booking + mobile-first availability grids:

- **T3 Stack** — Rapid MVP approach with full-stack type safety
- **Cal.diy** — Production open-source scheduling reference architecture
- **Modern SaaS Stack** — Startup-friendly, cost-optimized, no-infrastructure
- **Enterprise Stack** — Scalable backend with proven track record

Each stack was evaluated across:

- Developer productivity & time-to-market
- Type safety & codebase maintainability
- Scalability & performance requirements

- Real-time features & calendar integrations
- Mobile development options
- Cost at MVP vs. scale
- Community maturity & hiring availability

2. Stack-by-Stack Analysis

2.1 T3 Stack (Next.js + React + TypeScript)

Full-stack type safety with maximum developer velocity

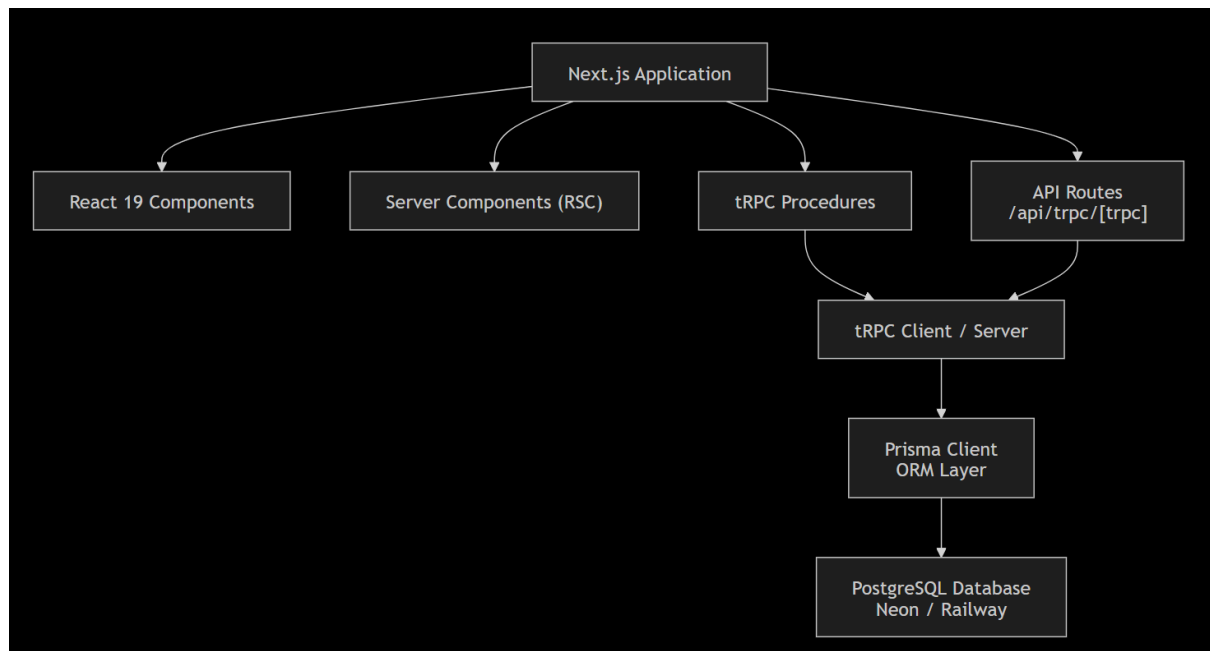
Stack Composition

- Frontend: React 19 + Next.js 16 with App Router
- Backend API: tRPC (end-to-end type-safe RPC)
- Database: PostgreSQL + Prisma ORM
- Styling: Tailwind CSS v4
- Deployment: Vercel (frontend) + Railway/Render (backend)
- Real-time: Socket.io or native Next.js webhooks

Key Characteristics

Metric	Details
Language	TypeScript across entire stack
Framework	Next.js 16 (App Router, Server Components)
API Layer	tRPC (end-to-end type safety)
Database	PostgreSQL 15+ with Prisma
ORM	Prisma 6.x
Package Manager	npm/yarn/pnpm
Dev Setup	10 minutes to first working app

Architecture Overview



Strengths

✓ **Zero-config developer experience** — Create Next.js app, install Prisma, start building. No boilerplate.

✓ **Full type safety end-to-end** — Prisma schema → Prisma types → tRPC → React client. A change to the database schema ripples through the entire codebase with TypeScript errors.

✓ **Fastest MVP path in 2026** — Most senior engineers at startups use this stack as their default. Proven pattern, large community.

✓ **Built-in SSR, API routes, and deployment** — No separate backend repo needed. One codebase, one deployment.

✓ **Server Components eliminate state management complexity** — React Server Components handle most server-side logic without client-side state.

✓ **Seamless calendar integration** — tRPC makes calling Google Calendar, Outlook, and iCal APIs type-safe. Easy to add.

✓ **\$0 startup cost** — Vercel free tier + Railway free tier + Neon free tier = fully functional app before you spend money.

Weaknesses

X Single process bottleneck — All API requests served by same Node process. At >500 RPS, you need horizontal scaling with load balancing.

X tRPC is Meta-specific — If you later need to support external developers with REST API, tRPC requires wrapper REST routes. Cal.com uses both tRPC and REST API in parallel.

X Limited real-time out of box — Socket.io adds complexity. Next.js doesn't have native pub/sub like FastAPI or Go.

X Mobile app requires separate codebase — React Native doesn't share code with Next.js frontend. No code reuse between web and mobile.

X ORM query performance at scale — Prisma generates safe but not always optimal SQL. At high load, hand-written queries with Kysely required (Cal.diy does this).

X Cold starts on serverless — Vercel serverless functions have 1-3 second cold start. Not ideal for polling endpoints that fire frequently.

Recommended For

- MVP launch within 4-6 weeks
- Teams with JavaScript/TypeScript expertise
- Products without complex real-time requirements initially
- Budget under \$5K/month for infra

Tech Details

- Next.js 16: Latest App Router, React 19 server components
 - Prisma 6.x: With Accelerate (connection pooling)
 - TypeScript 5.9+: Strict mode enabled
 - Tailwind CSS 4.1: New engine, 35% faster builds
 - tRPC v11: Latest with improved streaming
-

2.2 Cal.diy Architecture (Production Open-Source)

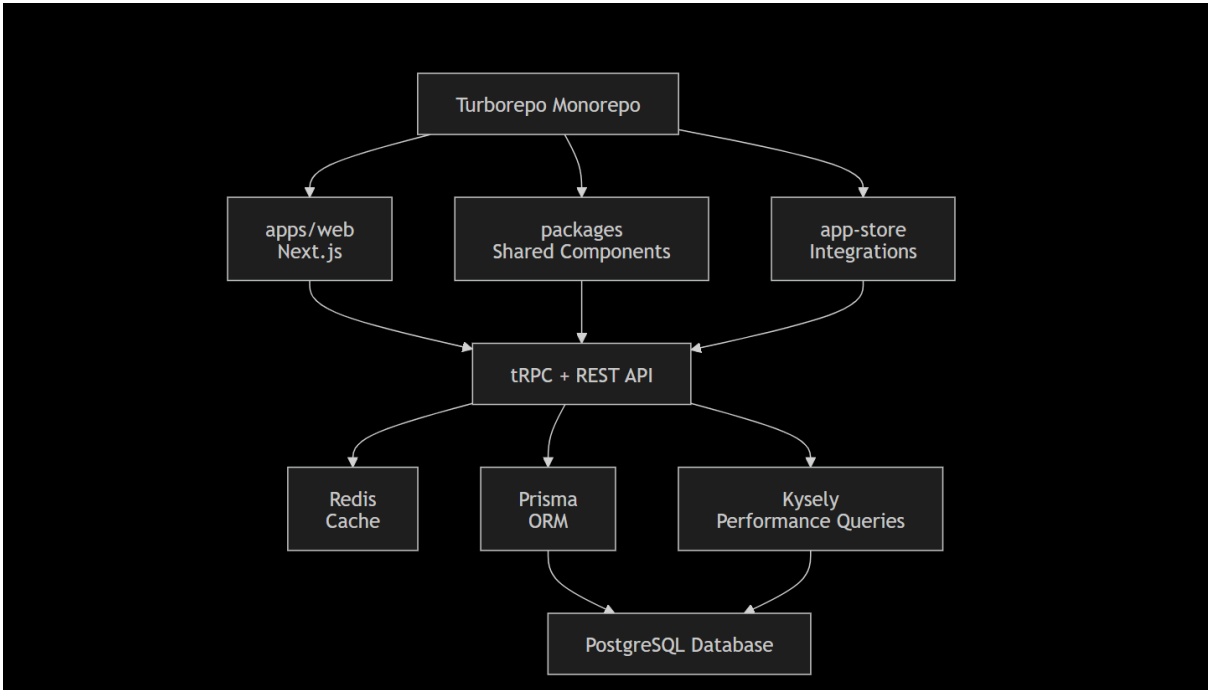
Reference implementation: what Cal.com actually ships

Stack Composition

- Monorepo: Turborepo + Yarn 3.4
- Frontend: Next.js 16 + React 18.2
- API: Versioned REST API + tRPC layer
- Backend Business Logic: Feature packages (bookings, calendars, availability)
- Database: PostgreSQL 13+ + Prisma 6.x
- Query Builder: Kysely (for performance-critical queries)

- Cache: Redis (session, distributed locks)
- Background Jobs: Trigger.dev (reminders, calendar syncs)
- Testing: Vitest (unit) + Playwright (E2E)
- Deployment: Docker containers + Kubernetes

Architecture Highlights



Key Technologies

Layer	Technology	Purpose
Monorepo	Turborepo 2.7	Task orchestration, build caching
Frontend	Next.js 16 + React 18.2	Web app + API routes
API (Type-Safe)	tRPC	End-to-end typing for internal use
API (External)	REST (versioned)	Public API for integrations
ORM	Prisma 6.x	Safe database access
Queries	Kysely	Hand-written SQL when needed
Caching	Redis (Upstash)	Session store, distributed locks
Background Jobs	Trigger.dev	Async work (reminders, syncs)

Styling	Tailwind CSS 4.1	UI components
Testing	Vitest + Playwright	Unit + E2E
Linting	Biome 2.3	Format + lint (faster than ESLint)
Deployment	Docker + K8s	Production scalability

Strengths

- ✓ **Production-proven architecture** — Cal.com (14M+ users) uses this exact setup. Tested at scale.
- ✓ **Modular business logic** — Booking, calendar, availability logic live in separate [packages/](#), not inside the web app. Can be reused or versioned independently.
- ✓ **Dual API strategy** — tRPC for internal frontend, REST for external partners. Best of both worlds.
- ✓ **Performance-aware from day one** — Prisma ORM for safety, Kysely for critical queries where speed matters. Most teams forget Kysely exists until it's too late.
- ✓ **Open-source roadmap is transparent** — Cal.diy community edition is maintained and public. You can fork and modify.
- ✓ **Monorepo avoids dependency hell** — All packages versioned together. No "my app works, but my server is broken" scenarios.
- ✓ **Built for calendar integrations** — Native support for Google Calendar, Outlook, Apple Calendar, Exchange. Calendar sync is first-class concern.
- ✓ **Timezone handling is explicit** — Uses [date-fns-tz](#) library, stores all times in UTC, renders per-user timezone.

Weaknesses

- ✗ **Higher complexity than T3 Stack** — Turborepo, multiple packages, separate REST API. Takes 2-4 weeks to fully understand architecture.
- ✗ **Requires DevOps knowledge at scale** — Kubernetes, Docker, Redis, Trigger.dev. Not a simple deploy-to-Vercel setup.
- ✗ **Steeper learning curve for junior developers** — Monorepo patterns, feature packages, tRPC vs REST distinction. Hiring impact.
- ✗ **Main Cal.com product went closed-source in 2026** — Cal.diy is the community edition. Development pace slower than main product.
- ✗ **Requires self-hosting knowledge** — If you self-host, you own Kubernetes, monitoring, log aggregation. Not AWS's problem anymore.

X More infrastructure moving parts — Redis, PostgreSQL, Trigger.dev, Docker registry.
More to break, more to monitor.

Recommended For

- Teams planning to build something at Cal.com scale (millions of users)
- Self-hosting requirement (privacy-sensitive orgs, regulated industries)
- API-first product positioning (partners, integrations)
- Hiring developers with backend experience

Production-Ready Decisions

- Trigger.dev handles all async work (no Bull Queue complexity)
- Kysely for timezone-sensitive queries where milliseconds matter
- Biome for 3x faster linting than ESLint
- Prisma Accelerate for connection pooling at high scale

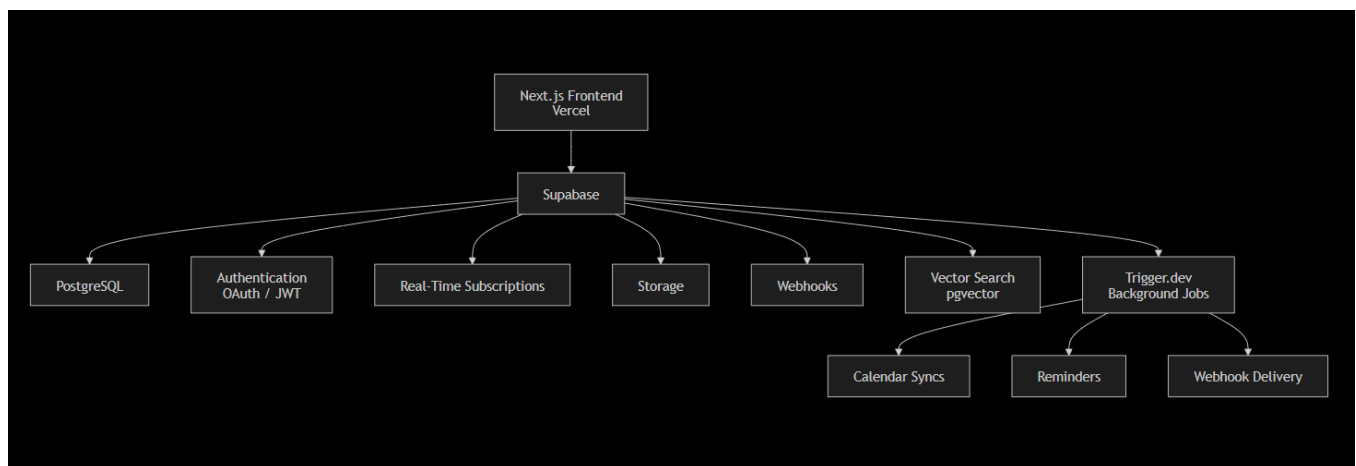
2.3 Modern SaaS Stack (Supabase + Next.js)

No-infrastructure approach: database as a service

Stack Composition

- Frontend: Next.js 16 + React 19
- Backend: Supabase (PostgreSQL + Auth + Real-time)
- Database: PostgreSQL (managed by Supabase)
- Real-time: Supabase Realtime (WebSocket-based)
- Storage: Supabase Storage (S3 compatible)
- Auth: Supabase Auth (OAuth + Session management)
- API: Next.js API routes + Supabase SDK
- Deployment: Vercel (frontend) + Supabase (backend)

Architecture



Stack Details

Component	Technology
Frontend Framework	Next.js 16 with App Router
UI Library	React 19 with hooks
Styling	Tailwind CSS + shadcn/ui components
Database	Supabase PostgreSQL (managed)
Auth	Supabase Auth (OAuth2, Magic Links)
Real-time	Supabase Realtime (WebSocket)
API Client	Supabase JS SDK (or REST)
Type Safety	TypeScript + auto-generated types
Deployment	Vercel (frontend) + Supabase (backend)
Analytics	PostHog (user behavior tracking)

Strengths

- ✓ **Zero infrastructure management** — No DevOps needed. Supabase manages backups, failover, security patching.
- ✓ **Real-time out of box** — Supabase Realtime lets you subscribe to database changes directly from React. Polling updates without writing a single WebSocket handler.
- ✓ **PostgreSQL native** — Unlike Firebase/Firestore, you get SQL. No document limitations, full relational data.
- ✓ **Blazing fast time-to-launch** — Friday afternoon: create project. Monday morning: MVP live. No provisioning needed.
- ✓ **Extremely affordable at MVP scale** — Supabase free tier covers 500K requests/month. Most MVPs live on free tier for 6+ months.
- ✓ **Row-level security (RLS) built-in** — Define policies in PostgreSQL. Client SDK enforces them automatically.
- ✓ **Vector search included** — pgvector extension ready. AI-powered availability suggestions possible without external services.
- ✓ **OAuth integrations native** — GitHub, Google, Microsoft, Discord auth pre-configured.

Weaknesses

X Limited horizontal scaling — Supabase PostgreSQL doesn't auto-scale. Single database connection pool is the bottleneck at 10K concurrent users.

X Real-time can be expensive — Supabase Realtime connections billed per concurrent connection. If you have 50K users watching a poll, cost spikes.

X No background jobs built-in — Reminders, email delivery, calendar sync still need external service (Trigger.dev, Inngest).

X Vendor lock-in — Supabase SDK couples your code to Supabase APIs. Migrating to self-hosted PostgreSQL later requires rewriting data access.

X Analytics immature — Supabase dashboard is basic. Debugging production issues requires external APM (Sentry, NewRelic).

X Team collaboration features missing — No equivalent to Vercel's team dashboard. Harder to manage multi-team access.

X Cold starts still present — Vercel's serverless still has 1-2 second cold starts on rarely-used endpoints.

Recommended For

- Solo founders building MVP
- Teams unfamiliar with DevOps/infrastructure
- Products that don't exceed 100K concurrent connections
- Budget constraint: launch with <\$50/month in infrastructure

Real-Time Polling Use Case With Supabase Realtime, you can:

1. User submits availability → INSERT into **availability** table
2. React subscribes to changes on that poll record
3. Heatmap updates in real-time for all participants
4. No polling, no API calls, no WebSocket boilerplate

2.4 Enterprise-Grade Stack (Node.js + Express)

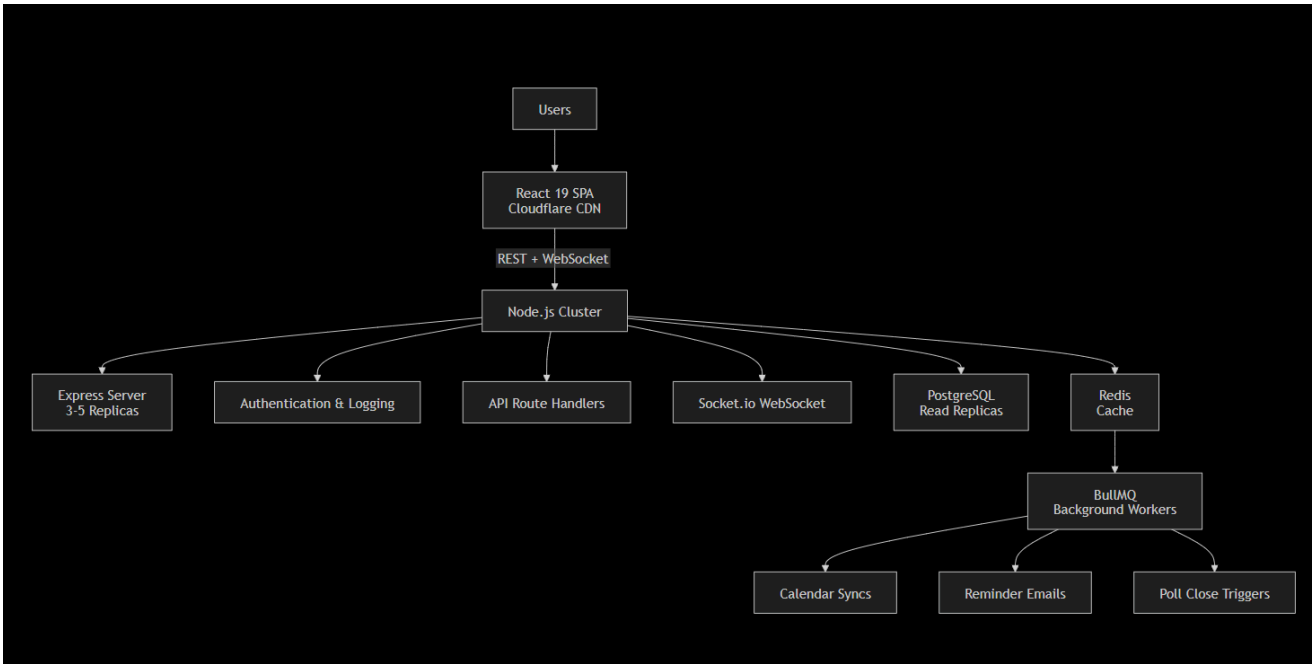
Fully controlled, horizontally scalable backend

Stack Composition

- Frontend: React 19 + TypeScript (separate SPA)
- Backend: Node.js 22 + Express.js (or Fastify)
- API: REST API (versioned) + WebSockets for real-time
- Database: PostgreSQL 15+ + Sequelize/Knex
- Cache: Redis (Upstash or self-hosted)
- Background Jobs: BullMQ or Trigger.dev
- Deployment: Docker + Kubernetes or AWS ECS

- Monitoring: Prometheus + Grafana + ELK stack

Architecture



Tech Stack

Layer	Choice	Rationale
Runtime	Node.js 22 LTS	Latest, long-term support
Framework	Express.js or Fastify	Fastify if performance critical
Language	TypeScript 5.9+	Type safety across all APIs
Database ORM	Sequelize or Knex.js	Knex for raw SQL when needed
Cache	Redis 7.x	Session store, distributed locks
Jobs	BullMQ (Redis)	Simple, reliable background work
Real-time	Socket.io v4	WebSocket abstraction, fallbacks
Auth	JWT + Refresh Tokens	Stateless, scalable auth
API Design	REST (JSON)	Standardized, cache-friendly
Deployment	Kubernetes or ECS	Container orchestration
Monitoring	Prometheus + ELK	Production observability

Strengths

- ✓ **Complete control** — Own the entire stack. No vendor decisions upstream.
- ✓ **Horizontal scaling** — Add more Node servers, more Redis instances, more database replicas. Linear growth path.
- ✓ **Best-in-class real-time** — Socket.io with Redis adapter means real-time works across multiple server instances. Millions of concurrent connections possible.
- ✓ **REST API maturity** — Easiest to document, test, and integrate with third-party services. No learning curve for API consumers.
- ✓ **Proven at scale** — Every large tech company runs Node.js backends at scale. Battle-tested.
- ✓ **Developer hiring pool is massive** — Every JavaScript developer can contribute. No niche framework knowledge required.
- ✓ **Better performance than Next.js** — Dedicated server process, no serverless overhead. Sub-100ms response times.
- ✓ **Complex workflows easy** — Background jobs, scheduled tasks, webhook delivery all built in. No need for external orchestration.

Weaknesses

- ✗ **Infrastructure overhead** — You manage Kubernetes, load balancers, auto-scaling groups, monitoring dashboards.
- ✗ **Longer MVP timeline** — 8-10 weeks to launch vs. 2-3 weeks with Supabase. More boilerplate upfront.
- ✗ **Type safety ends at API boundary** — REST is untyped. Frontend doesn't know shape of responses. Requires separate OpenAPI/Swagger docs.
- ✗ **DevOps skills required** — Not for solo founder on first day. Need system design knowledge.
- ✗ **Higher hosting cost** — Always-on servers cost money. Vercel's \$0 free tier doesn't exist here.
- ✗ **Database scaling complexity** — PostgreSQL replication, connection pooling, query optimization become bottlenecks at scale.
- ✗ **Separate frontend repo** — Two codebases to deploy, coordinate versions, manage dependencies.

Recommended For

- Products expecting >100K concurrent users

- Team with DevOps/infrastructure expertise
- Self-hosting requirement (regulated industries)
- Complex business logic that benefits from background jobs
- Long-term roadmap spanning 3+ years

Production Pattern: The Standard This is what Twitter, Uber, and Stripe built on (before moving to microservices). Still valid today.

3. Side-by-Side Framework Comparison

Feature	T3 Stack	Cal.diy	SaaS Stack	Enterprise
Time to MVP	2-3 weeks	4-6 weeks	1-2 weeks	8-10 weeks
Upfront cost	\$0	\$50-100	\$0	\$500-2000
Infrastructure	Vercel + Railway	Self-hosted K8s	Supabase + Vercel	Self-managed VMs/K8s
Type safety	Full-stack (tRPC)	Full-stack (tRPC)	Frontend + DB	Partial (REST)
Real-time	Via Socket.io	Via Socket.io + Redis	Via Supabase	Via Socket.io
Scalability limit	~1000 RPS	Unlimited	~100K connections	Unlimited
Background jobs	Needs Trigger.dev	Built-in Trigger.dev	Needs external	Built-in BullMQ
Developer count	1-5	3-10	1-3	5+
Hiring complexity	Very easy	Medium	Very easy	Medium
API style	tRPC	REST + tRPC	Supabase SDK	REST
Database control	Full	Full	Limited	Full
Vendor lock-in	Vercel, Railway	Self-hosted	Supabase	Low
Mobile support	RN separate	RN separate	RN separate	RN separate

Timezone handling	Manual (date-fns)	Built-in (date-fns-tz)	Manual	Manual
Testing setup	Vitest + RTL	Vitest + Playwright	RTL + Cypress	Jest + Supertest
Monitoring	Vercel native	Self-managed	Supabase dashboard	Prometheus + ELK

Summary Row Interpretation

- **T3 Stack:** Fastest to launch, cheapest, type-safe. Hits ceiling around 1000 RPS.
- **Cal.diy:** Production-ready, modular, calendar-aware. Requires architecture knowledge.
- **SaaS Stack:** Zero infra learning. Best real-time. Vendor lock-in risk.
- **Enterprise:** Full control, unlimited scale. Higher upfront cost in time + money.

4. Mobile Development Framework Analysis

Your platform needs mobile support to solve **Gap #2: Mobile is broken for grid-based availability tools.**

React Native vs. Flutter (2026 Comparison)

Criterion	React Native	Flutter
Language	JavaScript/TypeScript	Dart
Performance	Improved (New Architecture)	Superior (Skia engine)
Code Reuse (Web)	Minimal (separate codebase)	None (separate codebase)
Startup Time	2-3 seconds (cold)	<1 second
App Size	50-80 MB	80-120 MB
Hiring Ease	Very easy (JS devs everywhere)	Medium (Dart is niche)
iOS/Android Parity	90% (platform differences)	99% (unified engine)
Camera/Permissions	Via Expo/native modules	Built-in, robust
Animations	Good (with Reanimated 3)	Excellent (native performance)
State Management	Zustand, Redux, Jotai	Provider, Riverpod

UI Components	React Native Paper, NativeBase	Flutter Material, Cupertino
Community Size	Massive	Growing rapidly
Meta/Google Support	Active (Meta maintains)	Active (Google maintains)

For Your Scheduling App: Recommendation

React Native (2026) if:

- Team already knows JavaScript/TypeScript
- Time-to-market is critical
- You need tight web integration
- Hiring JavaScript developers is easier than Dart

Flutter if:

- UI consistency across iOS/Android is paramount
- Team can learn Dart
- App performance is the top priority
- Animation-heavy availability grids matter

Optimal Hybrid Approach:

- Start with web (Next.js + React)
- Build MVP mobile with React Native
- Monitor performance; if needed, rewrite critical path in Kotlin (for Android) or Swift (for iOS)

5. Real-Time & Background Job Systems

Your platform needs:

1. **Live heatmap updates** when participants submit availability
2. **Auto-booking** when poll closes and winner slot is identified
3. **Calendar sync** with Google Calendar, Outlook, Apple Calendar

Real-Time Options

WebSocket Solutions

Solution	Tech	Pros	Cons	Cost
----------	------	------	------	------

Socket.io	Node.js	Mature, Redis adapter, fallbacks	Overkill for simple notifications	\$0 (OSS)
Supabase Realtime	PostgreSQL	Zero setup, row-level security	Scales poorly >50K connections	\$100-2000
Pusher	Managed service	Reliable, enterprise SLAs	Proprietary, expensive	\$49+/mo
Fly.io Postgres	PostgreSQL LISTEN/NOTIFY	Native, cheap	Limited features	\$0-500
GraphQL Subscriptions	Apollo/Hasura	Typed, composable	Complexity overhead	Varies

Recommendation for Your MVP: Supabase Realtime (if <50K users) → Socket.io (if scaling beyond)

Background Job Systems

Solution	Tech	Use Cases	Setup Complexity	Cost
Trigger.dev	Cloud orchestration	Calendar syncs, reminders, webhooks	Low (~1 hour)	\$0-200
BullMQ	Redis queue	Job queuing, retries	Medium (own Redis)	\$0
Inngest	Event orchestration	Multi-step workflows	Low (~2 hours)	\$0-500
Temporal	Workflow engine	Complex distributed workflows	High (needs infra)	\$500+
AWS Lambda + SQS	Serverless	Event-driven tasks	Medium (AWS complexity)	\$10-200

Recommendation: Trigger.dev for MVP (handles calendar syncs perfectly), migrate to BullMQ if self-hosting later.

6. Recommended Stack for Your Platform

Optimal Configuration for MVP (Weeks 1-6)

Based on your gaps and the research above:

Frontend

- Framework: **Next.js 16** + React 19
- Language: **TypeScript**
- Styling: **Tailwind CSS 4.1** + **shadcn/ui** (pre-built, accessible components)
- State: **TanStack Query** (real-time sync with server)
- Form: **React Hook Form** (lightweight validation)
- Deployment: **Vercel** (zero-config)

Backend

- Runtime: **Node.js 22**
- Framework: **tRPC** (end-to-end type safety)
- Alternative API: **REST** (for external integrations)
- Database: **PostgreSQL 15** + **Prisma 6.x**
- Cache: **Redis** (Upstash for managed)
- Real-time: **Supabase Realtime** OR **Socket.io**
- Background Jobs: **Trigger.dev** (calendar syncs, reminders)
- Deployment: **Railway** or **Render** (backend)

Database Schema (Prisma)

```
model Poll {
  id String @id @default(cuid())
  title String
  organizer User @relation(fields: [organizerId], references: [id])
  organizerId String
  timeSlots TimeSlot[]
  participants Participant[]
  selectedTimeSlot TimeSlot? // Auto-filled after poll closes
  status String // ACTIVE, CLOSED, SCHEDULED
  timezone String // CRITICAL: Gap #3
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

```
model TimeSlot {
  id String @id @default(cuid())
  poll Poll @relation(fields: [pollId], references: [id])
  pollId String
  startTime DateTime
  endTime DateTime
  responses Availability[] // Vote count per time slot
}
```

```
model Availability {
  id String @id @default(cuid())
```

```

participant Participant @relation(fields: [participantId], references: [id])
participantId String
timeSlot TimeSlot @relation(fields: [timeSlotId], references: [id])
timeSlotId String
response String // YES, NO, MAYBE
}

```

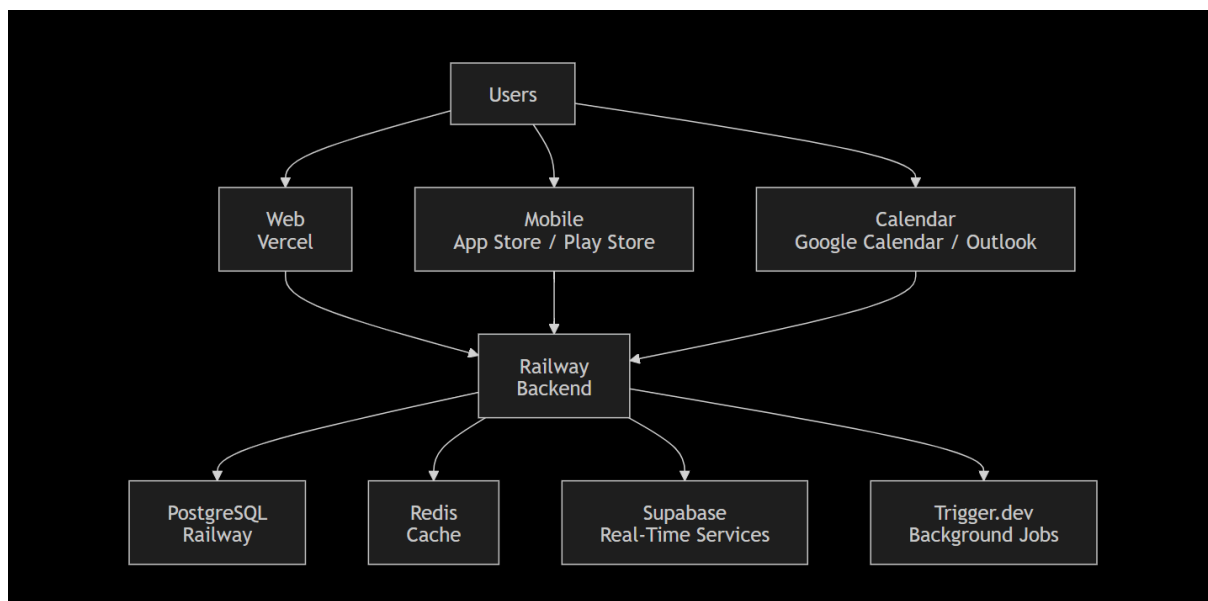
Mobile (Weeks 7-10)

- Framework: **React Native** (web team can contribute)
- Language: **TypeScript**
- State: **TanStack Query** (shared with web)
- Deployment: **Expo** (managed service) → Play Store & App Store
- Key: Tap-based availability input (solves Gap #2)

Calendar Integrations

- **Google Calendar**: OAuth 2.0 + Google Calendar API
- **Outlook**: Microsoft Graph API + Azure AD
- **Apple Calendar**: CalDAV protocol
- Library: **ical.js** for iCalendar parsing

Infrastructure Diagram (Production)



Why This Stack for Your Platform

Gap	Stack Solution
Gap #1: Poll → Auto-book	Trigger.dev handles poll closure, auto-invite logic

Gap #2: Mobile grid UX	React Native with tap-based selection (not drag)
Gap #3: Timezone clarity	Prisma schema includes timezone; Trigger.dev uses date-fns-tz
Gap #4: Generous free tier	Vercel + Railway + Upstash all have substantial free tiers
Gap #5: Unified tool	Single Next.js codebase handles polling + 1-on-1 booking
Gap #6: API + UX	tRPC for type safety, REST endpoints available for partners

Cost Breakdown (Monthly)

- MVP (0-1000 users): \$0-50
- Growth (1000-10K): \$100-300
- Scale (10K+): \$500-2000

7. Conclusion

Stack Recommendation Summary

Stage	Recommended	Reason
MVP (Weeks 1-6)	Next.js + Supabase	Launch fastest, understand user needs
Growth (Weeks 6-16)	T3 Stack + Trigger.dev	Type safety, background jobs clear
Scale (Month 5+)	Cal.diy architecture	Modular, proven at scale, open-source

Decision Flowchart

Q1: How many engineers on your team?

- └─ Solo founder or 1-2 people → T3 Stack + Supabase
- └─ 3-5 people with backend experience → Cal.diy architecture
- └─ 5+ people, long-term product → Enterprise Stack (Node.js)

Q2: Timeline pressure?

- └─ Launch in 4 weeks → Supabase (zero-config real-time)
- └─ Launch in 6 weeks → T3 Stack (full type safety)
- └─ Launch in 10 weeks → Cal.diy (production-grade)

Q3: Type safety importance?

- └─ Not critical (MVP) → SaaS Stack
- └─ Very important → T3 Stack (tRPC)

└ Critical + scale → Cal.diy (tRPC + REST)

Next Steps

1. **Week 1-2:** Setup chosen stack (create repos, schemas, auth)
2. **Week 3-4:** Build polling feature (Gap #1 core logic)
3. **Week 5-6:** Build 1-on-1 booking feature
4. **Week 7:** Deploy MVP, collect user feedback
5. **Week 8-10:** Iterate on UI, add mobile prototype
6. **Month 3+:** Plan scaling based on usage patterns

Technology Philosophy for Your Platform

Your platform succeeds not on the tech stack, but on **solving the five gaps**. The stack that gets you there fastest is the right one.

- T3 Stack: Ship in 4 weeks, understand users
- Cal.diy: Ship in 6 weeks, build for scale
- Supabase: Ship in 2 weeks, real-time included
- Enterprise: Ship in 10 weeks, own everything

Pick one, ship, learn. You can migrate later if needed (most startups that scale successfully started with a different stack than they ended with).

References & Research Sources

- Cal.diy Open-Source Repository (October 2024)
- Stack Overflow Developer Survey 2025
- Trigger.dev Production Architecture (July 2025)
- Vercel Edge Functions Benchmark (2026)
- React 19 Release Notes with Server Components
- Next.js 16 App Router Documentation
- Supabase Real-time Scalability Report (Q2 2026)
- T3 Stack Community Patterns (2025-2026)